Name:**Elvis Wachira** reg/no:**SCT211-0010/2025** course: **Computer science**

Recursion in Data Structures and Algorithms

Recursion refers to a technique where a function calls itself in order to solve a problem.

Instead of solving a problem all at once, recursion breaks it down into smaller , simpler subproblems that resemble the original problem. This makes it easier to manage complex tasks logically and systematically.

How Recursion Works;

Every recursive solution consists of two essential components: a base case and a recursive case.

The base case defines the simplest version of the problem that can be solved directly, without further recursion.

The recursive case reduces the problem toward the base case with each call. Without a well-defined base case, a recursive function would call itself indefinitely, resulting in a stack overflow.

Each call gets added to the call stack

Once the base case is hit, the stack unwinds and results get passed back up

No base case = infinite loop = stack overflow crash

An example is comuting the factorial of a number n:

```
factorial(0) = 1                  //base case
factorial(n) = n * factorial(n-1)  // recursive case
```

APPLICATION OF RECURSION

Recursion and Trees Structures

- Trees are naturally recursive - each node has subtrees that look just like the full tree
- This makes recursion the go-to approach for most tree operations like;
  - Inorder / preorder / postorder traversals
  - Finding the height of a tree
  - Searching for a value in a BST
  - Counting nodes
- Other structures that rely on recursion may include;;

- Tries - insert and search using recursive prefix steps
- Heaps - heapify is a recursive process
- Segment trees - range queries split recursively

Dividing and Conquering algorithms

- Many of most efficient algorithms use the recursive strategy to split a problem into smaller subproblems and slove each subproblem and combining the results

Examples include;

- Merge Sort:
  - Split the array in half, sort each half, merge them back
  - Time complexity: O(n log n) always

- Binary Search:
  - Check the midpoint, recurse left or right depending on the target
  - O(log n) - cuts the search space in half each time

Recursion vs Iteration

- Anything recursive can be rewritten with a loop and a manual stack
- Recursive code is usually cleaner and easier to read for tree and graph problems
- The downside of it is that deep recursion can blow the call stack
  - Python has a default limit of ~1,000 recursive calls

- Tail recursion: when the recursive call is the very last thing the function does
  - Some languages optimize this into a loop automatically
  - Python and Java do NOT do this by default

Memoization and Dynamic Programming

- Sometimes recursion solves the same subproblem many times which is quite wasteful
- Fibonacci is the classic example:
  - Naive recursion recalculates fib(2), fib(3) over and over
  - Time blows up to $O(2^n)$

- Memoization fixes this - cache each result the first time, reuse it after
    - Turns it into O(n) time
- This is called top-down dynamic programming
- Bottom-up DP does the same thing iteratively, filling a table from small to large